# 6 Things to know about AWS Elastic Beanstalk

Posted on August 20, 2016 by Ryan Fackett — No Comments ↓



ElasticBeanstalk is AWS' PaaS, and it's a powerful platform to accelerate application delivery in AWS' cloud. That said, I have come across a few things that have been stumbling blocks for some users of the service, especially as they try to deploy production workloads. In an effort to help you leverage beanstalk for production environments, I'll go through the most common issues and some workarounds.

## Security Groups



Beanstalk creates its own security groups so you don't have to. That's great if you don't want to create security groups, and an important concept to keep in mind. While you can certainly modify the ingress and egress rules of these security groups, what you can't easily do is use your own security groups. Beanstalk does allow you to add security groups to your EC2 instances through their interface, but you cannot remove the security group they created for the EC2 instances. This in contrast to AWS' OpsWorks service, which asks you if you want to use OpsWorks security groups or not. By saying no, you can use your own custom security groups. In order to use custom security groups with Beanstalk, you would:

- Create your custom security groups for the application layers
- Create your Beanstalk environment
- In the Beanstalk configuration, add your application security groups to the configuration
- Modify the ELB manually to use your security group instead of the Beanstalk security group (replacement)
- Manually remove all ingress and egress rules in the security group Beanstalk created for your EC2 instances

## Configuration Management


## Archives

- July 2017
- May 2017
- April 2017
- March 2017
- February 2017
- January 2017
- November 2016
- October 2016
- September 2016
- August 2016
- July 2016
- April 2016
- March 2016
- January 2016
- December 2015
- August 2015
- July 2015
- February 2015
- August 2014
- February 2014
- November 2013
- April 2013
- February 2013
- January 2013

## Categories

- Amazon Web Services
- AWS
- Azure
- Cloud
- Cloud Management
- Cost Optimization
- GCP
- General
- Health Care
- Private Cloud
- Public Cloud
- Security
- VMware

Beanstalk does provide a hook to modify the underlying EC2 instances on creation. Things like installing a log aggregation agent for example can be done through a feature called .ebextensions in your application code. Which brings us to .ebextensions in your application code. You are now bundling any configuration management in your application code. In the case where the development team is small, this might make sense (one place for all code). Once you have different teams managing system configuration and application features, they are now using a single application repository. Further, in order to test and deploy a system configuration change, you need to do an application release and test. If for example your application is Java, that means something as trivial as modifying the log aggregation agent configuration file entry means an application compilation, test and release.

```
~/workspace/my-app/
|-- .ebextensions
|   |-- environmentvariables.config
|   `-- healthcheckurl.config
|-- .elasticbeanstalk
|   `-- config.yml
|-- index.php
`-- styles.css
```

# Configuration Changes



Beanstalk has a wide set of configuration settings that provide a single place to make changes like the EC2 instance type, auto scaling group size, etc. However, not all changes just work. For example, moving from a single instance to a load balanced setup works well. Moving from a load balanced setup to a single instance, that can generate some errors. In any case though, it takes time to apply a change. Expect a significant amount of time for some environment changes, or just make sure you got the setup right the first time (for those values you are able to set during initial creation). Finally, most changes require the termination and recreation of the EC2 instances. This is a core concept for Beanstalk users to understand. The EC2 instances can and will be replaced as needed by Beanstalk. More on that later.

# Tagging

Beanstalk supports adding custom tags during environment creation. This is a great feature, especially for those users who have setup



Cost Allocation Tagging and want to accurately report on AWS fees by tag. However, once you set your tags, you cannot change them. There is no workflow to modify tags (keys or values) once the Beanstalk environment is created in Elastic Beanstalk. You would need to backtrace all those resources and modify the tagging manually (ELB, Auto Scaling Group, etc.).

## Custom AMI



Beanstalk supports using your Custom AMI (maybe you baked in your server changes into your custom AMI). That said, you can't use your custom AMI until after your environment is created. You have to build off the base Beanstalk AMI, wait for the environment to build, then replace the AMI with your custom one (thus rebuilding the environment to use the new AMI). Not terrible, but not terribly efficient either. Because Beanstalk uses agent software, you also need to follow a specific procedure to build a custom AMI that will function as expected in AWS.

## Server Changes



One really critical thing to always remember is, things you do on the EC2 instance manually are temporary changes. Any environment change or scaling activity can remove your changes since they are not part of your application code (configuration management). This replacement could be outside of your control, like maintenance by AWS for example. Users that like to SSH on to the server and make changes or view logs should consider this notion. If however you loathe SSH-ing on to your application servers, you may find this to be a benefit, not a hindrance. In general, in order to benefit from Elastic Beanstalk, your application should be developed with the thought that any configuration information needs to live in the Beanstalk environment, application, or .ebextensions. Any application state information needs to be kept somewhere other than the EC2 instance (Redis, Dynamo, etc.).